

Python Functions and Classes

Exercise Sheet

Sayan CHAKI

LIRIS – Ecole Centrale Lyon – Univ Lyon 2 – INSA

Learning goal

These exercises focus on constructors, instance variables, local and global variables, methods calling other methods, and variable-length arguments using `*args`. They intentionally avoid advanced OOP features.

Exercise 1 – Student Marks Manager

Create a class `StudentMarks` for storing and analysing the marks of one student.

1. Define a global variable `school_name` containing the name of the school.
2. Create `StudentMarks`. Its constructor `__init__(self, name)` must store the student's name and create an empty instance list `marks`.
3. Write `add_marks(self, *marks)`. It must accept any number of marks and add them to the instance's list.
4. Write `average(self)`. Return the average, or 0 if there are no marks. Use at least one local variable in the calculation.
5. Write `highest(self)`. Return the highest mark, or 0 if the list is empty.
6. Write `display(self)`. It must print the student's name, marks, average and highest mark. It should call `average()` and `highest()` rather than duplicate their calculations.
7. Write `school(self)` that reads and prints the global `school_name`.
8. Create two objects with different marks. Demonstrate that changing one student's marks does not change the other student's marks.

Example of intended use:

```
s1 = StudentMarks("Alice")
s1.add_marks(70, 85, 90)
s1.display()

s2 = StudentMarks("Bob")
s2.add_marks(60, 75, 80, 95)
s2.display()
```

Exercise 2 – Course Gradebook and Report Generator

A lecturer wants a small gradebook that can store several assessment groups and generate a final report. This exercise requires methods to cooperate rather than simply performing isolated arithmetic.

1. Define a global variable `PASS_MARK = 50`.

2. Create a class `CourseGradebook`. The constructor must receive `student_name` and `course_name`. It should create three empty instance lists: `homeworks`, `quizzes`, and `exams`.
3. Write `add_homeworks(self, *scores)`, `add_quizzes(self, *scores)`, and `add_exams(self, *scores)`. Each must accept a variable number of scores using `*args`.
4. Write a helper method `mean(self, *scores)` that returns the mean of any number of scores. Return 0 if no scores are supplied. Use a local variable such as `total`.
5. Write `category_averages(self)`. It must call `mean()` three times and return the homework, quiz, and exam averages.
6. Write `final_grade(self)`. It must call `category_averages()` and calculate

$$0.30 \times \text{HWAvg} + 0.20 \times \text{QuizAvg} + 0.50 \times \text{ExamAvg}.$$

Store intermediate values in local variables and return the final grade.

7. Write `status(self)`. It must call `final_grade()` and compare the result with the global `PASS_MARK`. Return "PASS" or "FAIL".
8. Write `report(self)`. It must call the other methods and print the student's name, course, category averages, final grade and status. Do not recalculate the formulas directly inside `report()`.
9. Write `add_assessment(self, category, *scores)`. Based on `category`, it must call one of `add_homeworks(*scores)`, `add_quizzes(*scores)`, or `add_exams(*scores)`. For an unknown category, print an error message.
10. Create two gradebooks. Add different numbers of scores to each object and generate both reports. Your code should demonstrate that instance variables belong to individual objects while `PASS_MARK` is shared globally.

Example calls (not a solution):

```
g1 = CourseGradebook("Alice", "Python")
g1.add_assessment("homework", 70, 80, 90, 85)
g1.add_assessment("quiz", 65, 88)
g1.add_assessment("exam", 74, 82)
g1.report()

g2 = CourseGradebook("Bob", "Python")
g2.add_assessment("homework", 45, 55, 62)
g2.add_assessment("quiz", 50)
g2.add_assessment("exam", 48, 57)
g2.report()
```

Challenge extension

Add `best_scores(self, *scores)` that returns both the highest score and the number of supplied scores. Then modify the report to show the best exam score by calling this helper. Handle the case where no scores exist.

Concept check

After completing the sheet, be able to explain the difference between: (1) a local variable such as `total`, (2) an instance variable such as `self.exams`, and (3) a global variable such as `PASS_MARK`. Also explain what Python places inside `scores` when a method is called with `*scores`, and why `self.mean(*self.exams)` uses the star again.